

TaskFlow

Diseño e implementación de un pipeline DevOps completo

Práctica Final · DevOps

Aplicación web con CI/CD, contenedores, pruebas automatizadas y monitoreo

Edwin De Los Santos · Matrícula 2025-1032

Contenido

1. README — visión general, arquitectura y API REST
2. Guía de instalación
3. Documentación del pipeline CI/CD
4. Manual de operaciones
5. Guion de la presentación

TaskFlow — Pipeline DevOps Completo

Práctica Final · Diseño e Implementación de un Pipeline DevOps Aplicación web de gestión de tareas con CI/CD, contenedores, pruebas automatizadas y monitoreo.

Autor: Edwin De Los Santos · Matrícula 2025-1032

 CI/CD Pipeline passing node 20.x docker multi-stage tests 54 passing coverage 92%

Tabla de contenidos

1. [Qué es este proyecto](#)
2. [Arquitectura](#)
3. [Stack tecnológico](#)
4. [Inicio rápido](#)
5. [Estructura del repositorio](#)
6. [API REST](#)
7. [Pruebas](#)
8. [Pipeline CI/CD](#)
9. [Monitoreo](#)
10. [Documentación adicional](#)

Qué es este proyecto

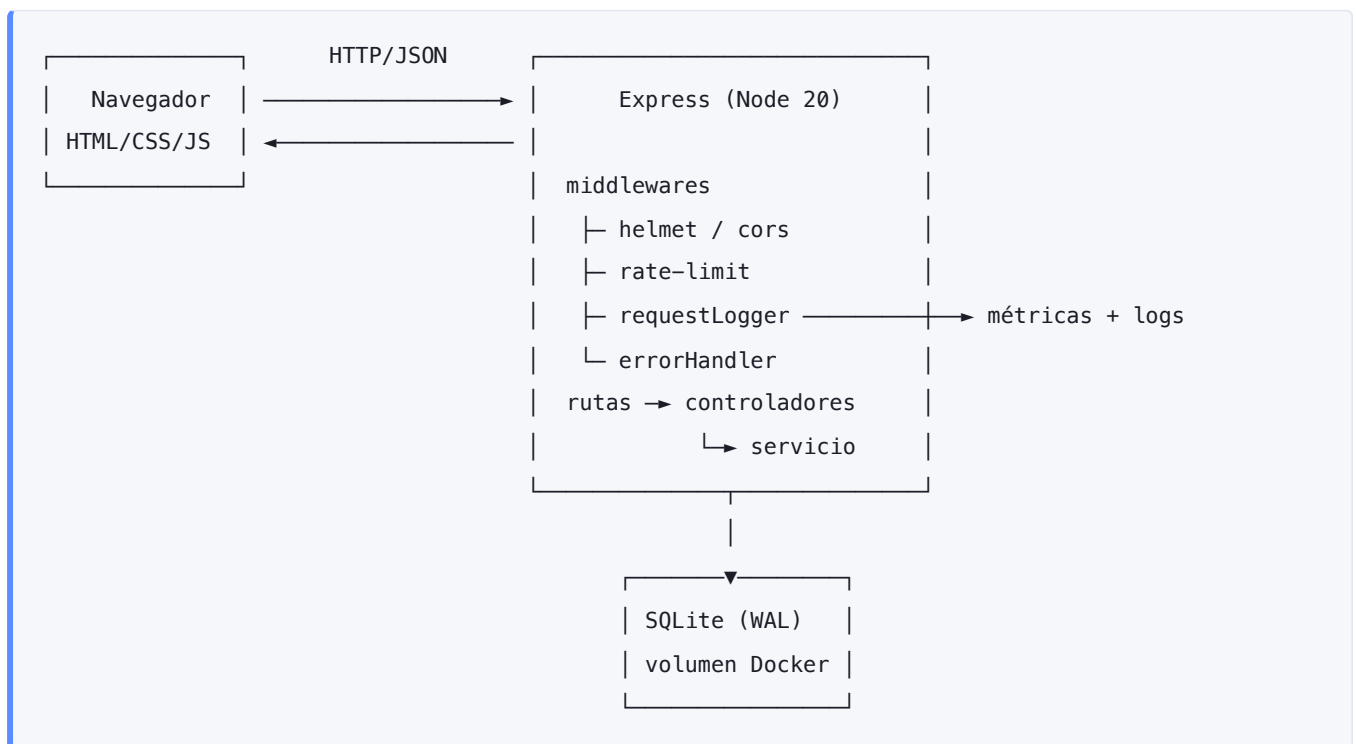
TaskFlow es un gestor de tareas sencillo (frontend + API REST + base de datos) que sirve de vehículo para demostrar un **pipeline DevOps completo y funcional**: desde el commit hasta el despliegue monitorizado.

El valor del proyecto no está en la complejidad de la aplicación, sino en todo lo que la rodea:

Requisito del enunciado	Implementación en este repositorio
Repositorio Git	Ramas <code>main</code> / <code>develop</code> , <code>.gitignore</code> , commits convencionales
Pipeline CI/CD	GitHub Actions con 8 jobs (.github/workflows/ci-cd.yml)
Contenedores Docker	Dockerfile multi-etapa + <code>docker-compose.yml</code> con 6 servicios
Pruebas automatizadas	54 pruebas Jest (unitarias + integración) + smoke tests en bash
Análisis estático	ESLint 9 (flat config) + Prettier + <code>npm audit</code>
Logs centralizados	Winston (JSON) → Promtail → Loki → Grafana
Métricas	<code>prom-client</code> → <code>/metrics</code> → Prometheus → dashboard Grafana
Alertas	6 reglas Prometheus enrutadas por Alertmanager

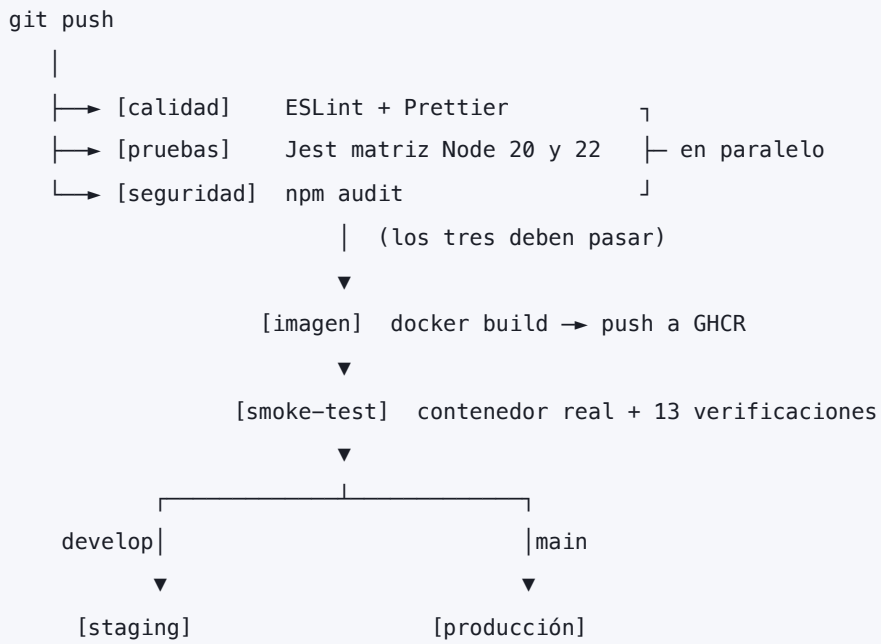
Arquitectura

Flujo de la aplicación

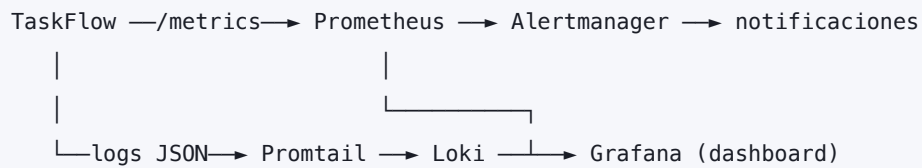


Arquitectura en capas: **rutas** → **controladores** → **servicio** → **base de datos**. Los controladores no ejecutan SQL y el servicio no conoce Express, lo que permite probar la lógica de negocio sin levantar un servidor HTTP.

Flujo del pipeline



Stack de monitoreo



Stack tecnológico

Capa	Tecnología	Motivo de la elección
Frontend	HTML5 + CSS3 + JavaScript ES5	Sin build step; CSP restrictiva posible
Backend	Node.js 20 + Express 4	Requisito del enunciado; ecosistema maduro
Base de datos	SQLite (better-sqlite3)	Cero servicios externos; API síncrona y rápida
Pruebas	Jest 29 + Supertest 7	Estándar de facto; cobertura integrada
Análisis estático	ESLint 9 + Prettier 3	Detecta errores antes de ejecutar
Contenedores	Docker multi-etapa + Compose	Imagen final mínima y sin root
CI/CD	GitHub Actions	Integrado al repositorio; sin infraestructura propia
Métricas	prom-client + Prometheus	Formato estándar de la industria
Logs	Winston + Promtail + Loki	JSON estructurado y consultable
Visualización	Grafana 11	Dashboards y logs en un solo lugar
Alertas	Alertmanager	Agrupación, inhibición y enrutado por severidad

Inicio rápido

Opción A — Stack completo con Docker (recomendado)

```
docker compose up -d --build
```

Servicio	URL	Credenciales
Aplicación TaskFlow	http://localhost:3000	—
Prometheus	http://localhost:9090	—
Grafana	http://localhost:3001	admin / admin
Alertmanager	http://localhost:9093	—
Loki (API)	http://localhost:3100	—

Detener y limpiar todo:

```
docker compose down -v
```

Opción B — Solo la aplicación, en local

```
npm install  
cp .env.example .env  
npm run dev
```

La aplicación queda en <http://localhost:3000>. Para cargar datos de ejemplo:

```
node scripts/seed.js
```

La [guía de instalación detallada](#) cubre requisitos previos, resolución de problemas y despliegue en un servidor.

Estructura del repositorio

```
DevOps-Final/
├── .github/workflows/ci-cd.yml    # Pipeline completo de GitHub Actions
├── src/
│   ├── app.js                   # Construcción de la app Express (sin listen)
│   ├── server.js                 # Arranque HTTP y apagado ordenado
│   ├── config/
│   │   ├── env.js               # Variables de entorno centralizadas
│   │   ├── logger.js            # Winston con salida JSON
│   │   └── metrics.js           # Registro de métricas Prometheus
│   ├── db/database.js           # Conexión SQLite, esquema y migraciones
│   ├── routes/                  # Definición de endpoints
│   ├── controllers/             # Adaptación HTTP ↔ servicio
│   ├── services/tasks.service.js # Lógica de negocio y validaciones
│   └── middlewares/             # Logging/métricas y manejo de errores
├── public/                      # Frontend estático (HTML, CSS, JS)
├── tests/
│   ├── unit/                   # Pruebas unitarias (sin HTTP)
│   └── integration/            # Pruebas de integración (Supertest)
├── monitoring/
│   ├── prometheus/             # Configuración y reglas de alerta
│   ├── alertmanager/           # Enrutado y agrupación de alertas
│   ├── loki/ promtail/         # Logs centralizados
│   └── grafana/                # Datasources y dashboard aprovisionados
├── scripts/
│   ├── smoke-test.sh           # 13 verificaciones contra una instancia viva
│   ├── wait-for-health.sh      # Espera activa del healthcheck
│   └── seed.js                 # Datos de ejemplo para la demo
├── docs/                        # Instalación, pipeline, operaciones, presentación
├── Dockerfile                  # Imagen multi-etapa
├── docker-compose.yml          # Stack app + monitoreo
├── docker-compose.prod.yml     # Override: despliega la imagen publicada
├── eslint.config.js  jest.config.js
└── README.md
```

API REST

Base: `http://localhost:3000`

Endpoints de negocio

Método	Ruta	Descripción	Códigos
GET	/api/tasks	Lista tareas. Query: <code>status=all active completed</code> , <code>priority=low medium high</code>	200
GET	/api/tasks/stats	Resumen agregado (total, completadas, pendientes, % avance)	200
GET	/api/tasks/:id	Obtiene una tarea	200, 400, 404
POST	/api/tasks	Crea una tarea	201, 400
PUT	/api/tasks/:id	Actualiza los campos enviados	200, 400, 404
PATCH	/api/tasks/:id/toggle	Invierte el estado completado	200, 404
DELETE	/api/tasks/:id	Elimina la tarea	200, 404

Endpoints de observabilidad

Método	Ruta	Descripción
GET	/health	Liveness probe: el proceso está vivo
GET	/health/ready	Readiness probe: verifica que SQLite responde (503 si no)
GET	/metrics	Métricas en formato Prometheus

Modelo de datos

```
{
  "id": 1,
  "title": "Configurar el workflow de GitHub Actions",
  "description": "Lint, pruebas, build y despliegue",
  "priority": "high",
  "completed": false,
  "createdAt": "2026-08-19 20:14:36",
  "updatedAt": "2026-08-19 20:14:36"
}
```

Reglas de validación (aplicadas en `src/services/tasks.service.js`):

- `title` : obligatorio, no vacío tras recortar espacios, máximo 120 caracteres.
- `description` : opcional, máximo 500 caracteres.
- `priority` : uno de `low` | `medium` | `high` (por defecto `medium`).
- `completed` : booleano estricto — `"true"` como texto se rechaza con 400.

Ejemplos

```
# Crear una tarea
curl -X POST http://localhost:3000/api/tasks \
  -H 'Content-Type: application/json' \
  -d '{"title":"Revisar el pipeline","priority":"high"}'

# Listar solo las pendientes
curl 'http://localhost:3000/api/tasks?status=active'

# Marcar como completada
curl -X PATCH http://localhost:3000/api/tasks/1/toggle

# Consultar el resumen
curl http://localhost:3000/api/tasks/stats
```

Formato de errores

Todas las respuestas de error comparten la misma estructura:

```
{
  "error": {
    "message": "El titulo es obligatorio.",
    "statusCode": 400,
    "requestId": "b3f1c2a4-..."
  }
}
```

El `requestId` viaja también en la cabecera `X-Request-Id` y aparece en los logs, lo que permite correlacionar un error reportado por un usuario con su traza exacta en Loki.

Pruebas

```
npm test          # Todas las suites (54 pruebas)
npm run test:unit  # Solo unitarias
npm run test:integration
npm run test:coverage # Con reporte y umbrales
npm run lint       # Análisis estático
```

Niveles de prueba

Nivel	Archivos	Qué cubre	Cantidad
Unitarias	tests/unit/tasks.service.test.js	Validaciones, reglas de negocio, consultas y ordenamiento	27
Unitarias	tests/unit/errorHandler.test.js	Manejo de 404 y enmascarado de errores 500	4
Integración	tests/integration/api.test.js	CRUD completo vía HTTP, filtros, ciclo de vida end-to-end	16
Integración	tests/integration/health.test.js	/health, /metrics, cabeceras de seguridad, 404	7
Smoke	scripts/smoke-test.sh	13 verificaciones contra un contenedor real en CI	—

Cobertura actual

File	% Stmts	% Branch	% Funcs	% Lines
-----	-----	-----	-----	-----
All files	92.60	81.48	96.77	92.90
src/services	94.68	92.53	100.00	95.69
src/middlewares	96.87	92.85	100.00	96.87
src/routes	91.89	100.00	100.00	91.89

Los umbrales mínimos están declarados en `jest.config.js` (70 % líneas, 60 % ramas). **Si la cobertura baja de ese umbral, el pipeline falla** — no es un reporte informativo.

Las pruebas usan **SQLite en memoria** (`tests/setupEnv.js` fija `DB_PATH=:memory:`), por lo que cada suite parte de una base limpia y no tocan datos reales.

Pipeline CI/CD

El pipeline vive en `.github/workflows/ci-cd.yml` y se dispara con:

- `push` a `main` o `develop`
- `pull_request` hacia `main` o `develop`
- etiquetas `v*` (releases)
- ejecución manual (`workflow_dispatch`)

#	Job	Qué hace	Bloquea si falla
1	calidad	ESLint + verificación de formato Prettier	Sí
2	pruebas	Jest en Node 20 y 22 + cobertura con umbrales	Sí
3	seguridad	<code>npm audit --audit-level=high</code>	Sí
4	imagen	Build multi-etapa y push a GHCR con etiquetas semánticas	Sí
5	smoke-test	Levanta el contenedor y ejecuta 13 verificaciones reales	Sí
6	desplegar-staging	Despliegue automático desde <code>develop</code>	—
7	desplegar-produccion	Despliegue desde <code>main</code> (con environment protegido)	—
8	resumen	Tabla de resultados en la interfaz de GitHub	No

Decisiones destacadas, explicadas en detalle en [docs/PIPELINE.md](#):

- Los jobs 1–3 corren **en paralelo**: el feedback de fallo llega en ~1 minuto en lugar de ~3.
- `concurrency` con `cancel-in-progress` evita desplegar un commit ya superado.
- Los **pull requests validan pero no publican** imágenes ni despliegan.
- El caché de `npm` y de `buildx` reduce el tiempo de las ejecuciones sucesivas.

Monitoreo

Métricas expuestas

Métrica	Tipo	Uso
<code>http_requests_total{method,route,status_code}</code>	Counter	Tráfico y tasa de error
<code>http_request_duration_seconds{...}</code>	Histogram	Percentiles p50 / p95 / p99
<code>http_errors_total{...}</code>	Counter	Alertas de errores
<code>taskflow_tasks_created_total</code>	Counter	Métrica de negocio
<code>taskflow_tasks_completed_total</code>	Counter	Métrica de negocio
<code>taskflow_tasks_in_database{status}</code>	Gauge	Estado actual de los datos
<code>taskflow_process_*</code>	Varios	CPU, memoria, event loop, handles

Las etiquetas de ruta están **normalizadas** (`/api/tasks/:id` en lugar de `/api/tasks/42`) para evitar la explosión de cardinalidad que rompería a Prometheus.

Alertas configuradas

Alerta	Condición	Espera	Severidad
AplicacionCaída	up == 0	1 min	critical
TasaDeErroresAlta	> 5 % de respuestas 5xx	2 min	critical
ExcesoDeErroresDeCliente	> 5 errores 4xx por segundo	5 min	warning
LatenciaAlta	p95 > 1 s	3 min	warning
MemoriaElevada	RSS > 400 MB	5 min	warning
ReinicioDelServicio	uptime < 5 min	inmediata	info

Alertmanager **agrupa** por `alertname` + `service`, enruta las críticas a un receptor con reenvío cada 30 minutos, e **inhibe** las alertas de latencia y memoria cuando el servicio ya está reportado como caído (evita la avalancha de avisos redundantes).

Logs centralizados

Winston emite JSON con campos fijos (`service`, `version`, `instance`, `env`) más el `requestId` de correlación. Promtail parsea ese JSON y promueve a etiqueta de Loki solo los campos de **baja cardinalidad** (`level`, `route`, `statusCode`); `requestId` y `durationMs` quedan como campos del log para no inflar el índice.

Consultas útiles en Grafana → Explore → Loki:

```
{job="taskflow"} | json | level="error"
{job="taskflow"} | json | durationMs > 500
{job="taskflow"} | json | requestId="b3f1c2a4-..."
```

Documentación adicional

Documento	Contenido
docs/INSTALACION.md	Requisitos, instalación paso a paso, despliegue y resolución de problemas
docs/PIPELINE.md	Anatomía del pipeline, decisiones de diseño y cómo extenderlo
docs/OPERACIONES.md	Manual de operaciones: runbooks, backups, rollback y checklists
docs/PRESENTACION.md	Guion de la presentación, demo en vivo y lecciones aprendidas

Licencia

MIT — proyecto académico.

Guía de Instalación — TaskFlow

Esta guía cubre desde la puesta en marcha en una máquina de desarrollo hasta el despliegue del stack completo en un servidor.

1. Requisitos previos

Para ejecución local (sin Docker)

Software	Versión mínima	Verificar con
Node.js	20.x	<code>node -v</code>
npm	10.x	<code>npm -v</code>
Git	2.x	<code>git --version</code>

Node 20 o superior es obligatorio. El proyecto usa `node:test` -era APIs de Node moderno y la dependencia nativa `better-sqlite3` publica binarios precompilados a partir de esa versión.

Para el stack completo

Software	Versión mínima	Verificar con
Docker Engine	24.x	<code>docker --version</code>
Docker Compose	v2 (plugin)	<code>docker compose version</code>

En macOS y Windows basta con **Docker Desktop**; asegúrese de que esté **en ejecución** antes de lanzar los comandos (`docker info` debe responder sin error).

Recursos recomendados

El stack completo levanta 6 contenedores. Asigne a Docker al menos:

- **4 GB de RAM**
- **2 CPU**
- **10 GB de disco** (Prometheus retiene 15 días y Loki 7)

2. Obtener el código

```
git clone https://github.com/edwindellossantos08/DevOps-Final.git
cd DevOps-Final
```

3. Instalación local (desarrollo)

3.1 Instalar dependencias

```
npm install
```

`better-sqlite3` intentará descargar un binario precompilado. Si su plataforma no tiene uno, lo compilará desde el código fuente, para lo cual necesita:

- **Linux:** `sudo apt-get install -y python3 make g++`
- **macOS:** `xcode-select --install`
- **Windows:** `npm install --global windows-build-tools` (como administrador)

3.2 Configurar las variables de entorno

```
cp .env.example .env
```

Variable	Por defecto	Descripción
<code>NODE_ENV</code>	<code>development</code>	Entorno: <code>development</code> , <code>test</code> o <code>production</code>
<code>PORT</code>	<code>3000</code>	Puerto HTTP de la API
<code>DB_PATH</code>	<code>./data/taskflow.db</code>	Ruta del archivo SQLite
<code>LOG_LEVEL</code>	<code>info</code>	<code>error</code> , <code>warn</code> , <code>info</code> , <code>http</code> o <code>debug</code>
<code>LOG_DIR</code>	<code>./logs</code>	Directorio de logs en JSON
<code>RATE_LIMIT_WINDOW_MS</code>	<code>60000</code>	Ventana del rate limiting
<code>RATE_LIMIT_MAX</code>	<code>300</code>	Peticiones máximas por ventana
<code>INSTANCE_ID</code>	<code>local</code>	Identificador de la instancia en métricas y logs

El archivo `.env` está en `.gitignore` y **nunca debe subirse al repositorio**.

3.3 Arrancar

```
npm run dev    # con recarga automática (node --watch)
npm start     # sin recarga
```

Abra <http://localhost:3000>. La base de datos y el directorio de logs se crean solos en el primer arranque.

3.4 Cargar datos de ejemplo (opcional)

```
node scripts/seed.js
```

Inserta 9 tareas que representan las fases del propio proyecto — útil para la demo y para que los paneles de Grafana muestren datos desde el primer minuto.

3.5 Verificar la instalación

```
npm test                # 54 pruebas deben pasar
npm run lint            # sin errores
./scripts/smoke-test.sh http://localhost:3000
```

4. Instalación con Docker (solo la aplicación)

```
# Construir la imagen
docker build --target runtime -t taskflow:local .

# Ejecutar con persistencia de datos
docker run -d \
  --name taskflow \
  -p 3000:3000 \
  -v taskflow-data:/app/data \
  -v taskflow-logs:/app/logs \
  taskflow:local

# Verificar el estado del healthcheck
docker ps --filter name=taskflow
docker logs -f taskflow
```

Notas sobre la imagen:

- Es **multi-etapa**: la etapa `deps` compila las dependencias nativas y la etapa `runtime` solo copia `node_modules` ya construido, `src/` y `public/`.
- Se ejecuta como el usuario **no privilegiado** `node`.
- Usa **tini** como PID 1 para que `SIGTERM` llegue a Node y el apagado sea ordenado.
- Incluye un **HEALTHCHECK** nativo que consulta `/health` cada 30 s.

5. Instalación del stack completo (aplicación + monitoreo)

```
docker compose up -d --build
```

Esto levanta 6 servicios:

Contenedor	Imagen	Puerto host	Función
taskflow-app	build local	3000	Aplicación
taskflow-prometheus	prom/prometheus:v2.54.1	9090	Recolección de métricas
taskflow-alertmanager	prom/alertmanager:v0.27.0	9093	Enrutado de alertas
taskflow-loki	grafana/loki:3.1.1	3100	Almacén de logs
taskflow-promtail	grafana/promtail:3.1.1	—	Envío de logs a Loki
taskflow-grafana	grafana/grafana:11.2.0	3001	Dashboards

5.1 Verificar que todo arrancó

```
docker compose ps          # todos en estado running/healthy
curl http://localhost:3000/health # {"status":"ok",...}
curl http://localhost:9090/-/healthy # Prometheus Server is Healthy
curl http://localhost:3100/ready  # ready
```

5.2 Comprobar que Prometheus ve la aplicación

Abra <http://localhost:9090/targets> — el target `taskflow-app` debe aparecer **UP**.

5.3 Abrir el dashboard

1. Entre a <http://localhost:3001> con `admin / admin`.
2. El datasource y el dashboard ya están **aprovisionados**: menú **Dashboards** → carpeta **TaskFlow** → **TaskFlow — Visión General**.
3. Genere tráfico para ver movimiento en los paneles:

```
for i in $(seq 1 50); do
  curl -s -X POST http://localhost:3000/api/tasks \
    -H 'Content-Type: application/json' \
    -d '{"title":"Carga $i"}' > /dev/null
  curl -s http://localhost:3000/api/tasks > /dev/null
done
```

5.4 Cambiar las credenciales de Grafana

Las credenciales por defecto son solo para la demo. Para cambiarlas:

```
GRAFANA_USER=miusuario GRAFANA_PASSWORD='una-clave-larga' docker compose up -d grafana
```

5.5 Detener el stack

```
docker compose down      # detiene y borra los contenedores, conserva los volúmenes
docker compose down -v   # además borra los volúmenes (se pierden datos y métricas)
```

6. Despliegue en un servidor

6.1 Preparar el servidor

```
# Instalar Docker (Ubuntu/Debian)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker "$USER" # requiere volver a iniciar sesión
```

6.2 Desplegar desde la imagen publicada por el pipeline

El pipeline publica la imagen en GitHub Container Registry:

```
echo "$GHCRTOKEN" | docker login ghcr.io -u edwindelossantos08 --password-stdin
docker pull ghcr.io/edwindelossantos08/devops-final:latest

docker run -d \
  --name taskflow \
  --restart unless-stopped \
  -p 3000:3000 \
  -e NODE_ENV=production \
  -e INSTANCE_ID=prod-1 \
  -v taskflow-data:/app/data \
  -v taskflow-logs:/app/logs \
  ghcr.io/edwindelossantos08/devops-final:latest
```

6.3 Actualizar a una versión nueva

En el servidor se usa el override `docker-compose.prod.yml`, que reemplaza la construcción local por la imagen ya publicada y verificada por el pipeline:

```
export TASKFLOW_IMAGE=ghcr.io/edwindelossantos08/devops-final:latest
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Compose reemplaza los contenedores cuya imagen cambió y **conserva los volúmenes**, por lo que no se pierden datos. Para volver a una versión anterior basta con fijar `TASKFLOW_IMAGE` a la etiqueta por SHA de ese commit (véase el [procedimiento de rollback](#)).

6.4 Proxy inverso con TLS (recomendado en producción)

La aplicación no termina TLS. Colóquela detrás de Nginx o Caddy:

```
server {
    listen 443 ssl http2;
    server_name taskflow.example.com;

    ssl_certificate      /etc/letsencrypt/live/taskflow.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/taskflow.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host          $host;
        proxy_set_header X-Real-IP      $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # /metrics no debería quedar expuesto públicamente
    location /metrics {
        allow 10.0.0.0/8;
        deny all;
        proxy_pass http://127.0.0.1:3000;
    }
}
```

La aplicación ya tiene `trust proxy` activado, por lo que `req.ip` reflejará la IP real del cliente y el `rate limiting` funcionará correctamente detrás del proxy.

7. Resolución de problemas

Síntoma	Causa probable	Solución
Cannot connect to the Docker daemon	Docker Desktop no está iniciado	Ábralo y espere a que <code>docker info</code> responda
EADDRINUSE: port 3000	Otro proceso usa el puerto	<code>lsof -i :3000</code> y termine el proceso, o use <code>PORT=3001 npm start</code>
node-gyp falla al instalar	Faltan herramientas de compilación	Instale <code>python3</code> , <code>make</code> y <code>g++</code> (ver §3.1)
SQLITE_CANTOPEN	El proceso no puede escribir en <code>DB_PATH</code>	Verifique permisos del directorio; en Docker, que el volumen esté montado
Prometheus muestra el target DOWN	La app no arrancó o no está en la misma red	<code>docker compose logs app</code> y confirme que ambos usan <code>taskflow-net</code>
Grafana sin datos	El datasource no resuelve Prometheus	Compruebe que el <code>uid</code> del datasource es <code>prometheus</code> (está fijado en el aprovisionamiento)
Loki sin logs	Promtail no ve el volumen de logs	Confirme que <code>taskflow-logs</code> está montado en ambos contenedores
Las pruebas fallan con SQLITE_BUSY	Una instancia local tiene la base bloqueada	Las pruebas usan <code>:memory:</code> ; verifique que <code>NODE_ENV=test</code> esté aplicado
429 Too Many Requests	Se superó el rate limit	Suba <code>RATE_LIMIT_MAX</code> o espere a que pase la ventana

Comandos de diagnóstico

```
docker compose ps          # estado y salud de cada servicio
docker compose logs -f app  # logs en vivo de la aplicación
docker compose logs --tail=100 prometheus
docker stats               # consumo de CPU y memoria
docker exec -it taskflow-app sh  # shell dentro del contenedor
curl -s localhost:3000/metrics | head # verificar que las métricas salen
```

8. Desinstalación

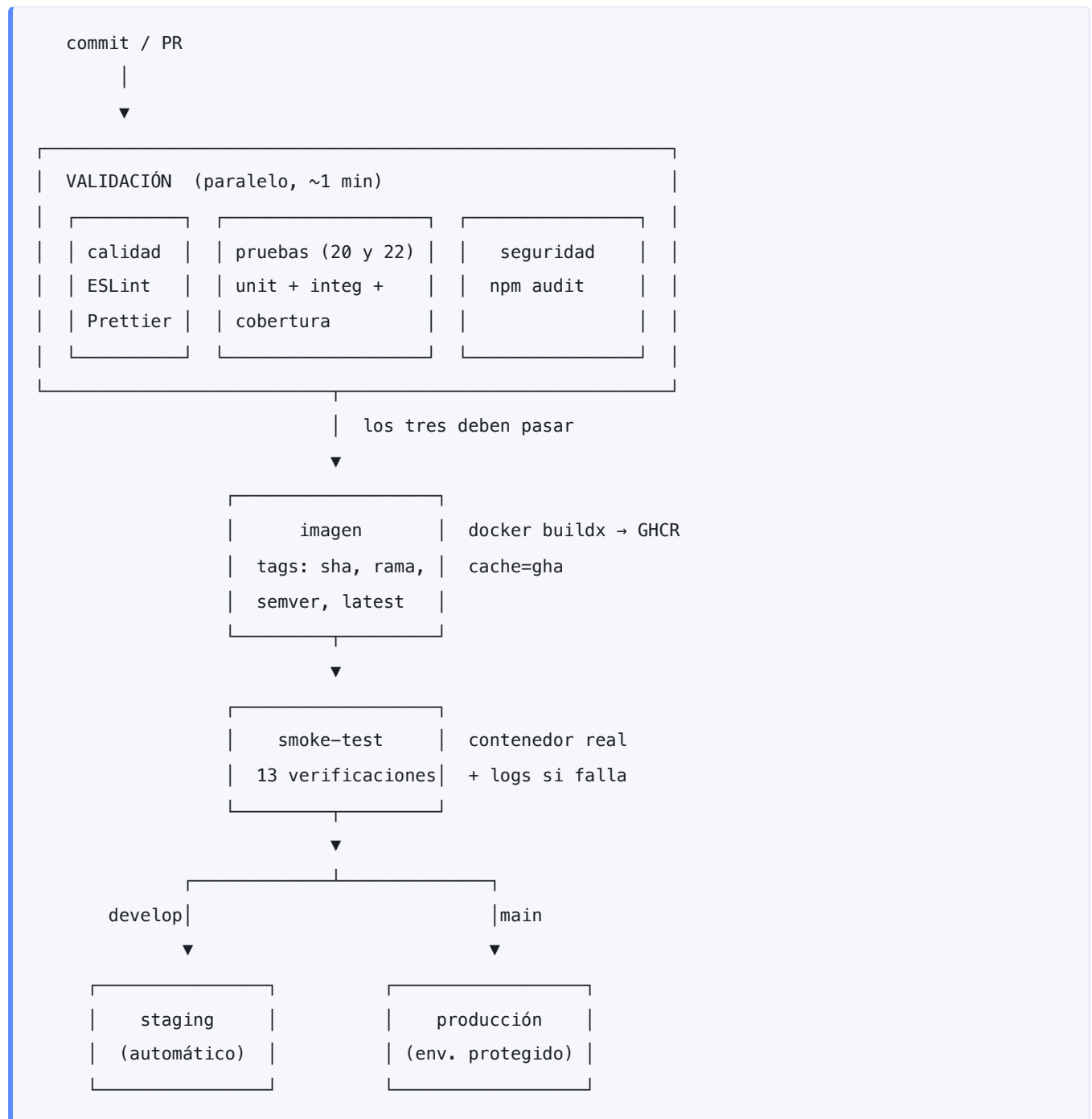
```
docker compose down -v      # contenedores, redes y volúmenes
docker rmi taskflow:local   # imagen local
rm -rf node_modules data logs coverage # artefactos locales
```

Documentación del Pipeline CI/CD

Archivo de definición: [.github/workflows/ci-cd.yml](#)

1. Visión general

El pipeline transforma un commit en una imagen desplegada y verificada, atravesando seis puertas de calidad. Cada puerta puede detener la entrega.



2. Disparadores

Evento	Ramas / patrones	Qué se ejecuta
push	main , develop	Pipeline completo, incluido despliegue
push	tags v*	Pipeline completo con etiquetado semántico de la imagen
pull_request	hacia main , develop	Solo validación: no construye ni despliega
workflow_dispatch	manual	Pipeline completo desde la interfaz de GitHub

Por qué los PR no publican imágenes

Un pull request es código **propuesto**, no aceptado. Construir y publicar una imagen desde una rama de terceros permitiría inyectar artefactos en el registro antes de la revisión. El condicional `if: github.event_name != 'pull_request'` en el job `imagen` cierra esa puerta.

3. Control de concurrencia

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

Si se empujan tres commits seguidos a `main`, las dos primeras ejecuciones se cancelan. Sin esto podría ocurrir una **condición de carrera de despliegue**: el pipeline del commit antiguo termina después del nuevo y deja producción en una versión atrasada.

4. Etapas en detalle

4.1 calidad — Análisis estático

```
- run: npm run lint # ESLint 9 (flat config)
- run: npm run format:check # Prettier en modo verificación
```

ESLint está configurado en `eslint.config.js` con dos contextos:

Contexto	Archivos	Globals	Particularidad
Servidor	src/ , tests/ , scripts/	Node + Jest	no-var , prefer-const , eeqeq como error
Navegador	public/	Browser	ES5 permitido: no-var desactivado

Reglas elevadas a **error** (rompen la build): `no-unused-vars`, `no-undef`, `eqeqeq`, `no-var`, `prefer-const`, `no-throw-literal`. Las de estilo puro se delegan a Prettier mediante `eslint-config-prettier`, que se aplica al final para desactivar cualquier regla en conflicto.

4.2 pruebas — Matriz de versiones

```
strategy:
  fail-fast: false
matrix:
  node: ['20', '22']
```

Se prueba contra la versión de producción (20 LTS) y contra la siguiente (22), lo que anticipa roturas antes de la migración. `fail-fast: false` permite ver el resultado de ambas: saber que *solo* falla en 22 es un diagnóstico muy distinto a que falle en las dos.

El job ejecuta tres pasos separados — unitarias, integración y cobertura — en lugar de uno solo. Al fallar, la interfaz de GitHub señala **qué nivel** de prueba se rompió sin tener que abrir los logs.

La cobertura se publica como artefacto (`actions/upload-artifact`) con retención de 7 días.

4.3 seguridad — Auditoría de dependencias

```
- run: npm audit --audit-level=high
```

Falla ante vulnerabilidades **high** o **critical**. Las moderadas y bajas se reportan sin bloquear: un umbral demasiado estricto genera fatiga de alertas y termina ignorándose.

Cómo extenderlo: añadir un escaneo de la imagen con Trivy es un paso natural:

```
- name: Escanear la imagen con Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: taskflow:ci
    severity: HIGH,CRITICAL
    exit-code: '1'
```

4.4 imagen — Construcción y publicación

```
needs: [calidad, pruebas, seguridad]
```

La dependencia declarada garantiza que **no existe una imagen publicada que no haya pasado las tres validaciones**. Es la propiedad central del pipeline.

`docker/metadata-action` genera automáticamente las etiquetas:

Patrón	Ejemplo resultante	Cuándo
<code>type=ref,event=branch</code>	<code>ghcr.io/...:main</code>	Siempre
<code>type=semver,pattern={{version}}</code>	<code>ghcr.io/...:1.2.0</code>	Al empujar el tag <code>v1.2.0</code>
<code>type=sha,format=short</code>	<code>ghcr.io/...:a1b2c3d</code>	Siempre — permite rollback exacto
<code>type=raw,value=latest</code>	<code>ghcr.io/...:latest</code>	Solo en la rama por defecto

La etiqueta por SHA es la que hace posible el rollback preciso descrito en [OPERACIONES.md](#).

Caché de capas:

```
cache-from: type=gha
cache-to: type=gha,mode=max
```

Si `package*.json` no cambió, la capa de `npm ci` se reutiliza y el build baja de varios minutos a segundos.

4.5 `smoke-test` — Verificación sobre el contenedor real

Esta etapa existe porque **las pruebas de Jest validan el código, no el artefacto**. Un `Dockerfile` puede olvidar copiar `public/`, fijar mal una variable de entorno o romper los permisos del volumen: nada de eso lo detecta una prueba unitaria.

Secuencia:

1. `docker build --target runtime -t taskflow:ci .`
2. `docker run -d -p 3000:3000 taskflow:ci`
3. `./scripts/wait-for-health.sh http://localhost:3000/health 60` — espera activa
4. `./scripts/smoke-test.sh http://localhost:3000` — 13 verificaciones
5. `if: failure()` → `docker logs taskflow-ci` — sin esto, un fallo aquí sería opaco
6. `if: always()` → `docker rm -f` — limpieza garantizada

Las verificaciones cubren: salud, readiness, métricas, frontend estático, listado, alta, validación de entrada, obtención, alternado, borrado, 404 tras borrado y ruta inexistente.

4.6 Despliegues

Rama	Entorno	Aprobación
<code>develop</code>	<code>staging</code>	Automática
<code>main</code>	<code>production</code>	Configurable como <i>required reviewer</i> en el repositorio

Los jobs usan la clave `environment:` de GitHub, que aporta dos cosas: la URL visible en la interfaz y la posibilidad de exigir **aprobación manual** y secretos propios por entorno sin tocar el YAML.

En este proyecto académico los pasos de despliegue imprimen el comando equivalente (`docker compose pull && docker compose up -d`) en lugar de ejecutar SSH contra un servidor real. La estructura del pipeline es la definitiva; solo se sustituye el contenido de ese paso por, por ejemplo:

```
- name: Desplegar por SSH
  uses: appleboy/ssh-action@v1
  with:
    host: ${ secrets.DEPLOY_HOST }
    username: ${ secrets.DEPLOY_USER }
    key: ${ secrets.DEPLOY_SSH_KEY }
    script: |
      cd /opt/taskflow
      docker compose pull
      docker compose up -d
      ./scripts/smoke-test.sh http://localhost:3000
```

4.7 resumen

Se ejecuta con `if: always()` y escribe una tabla en `$GITHUB_STEP_SUMMARY` con el resultado de cada etapa. Aparece en la portada de la ejecución, sin necesidad de desplegar logs.

5. Gestión de secretos

Secreto	Uso	Origen
<code>GITHUB_TOKEN</code>	Autenticación en GHCR	Provisto automáticamente por Actions
<code>DEPLOY_HOST</code> , <code>DEPLOY_USER</code> , <code>DEPLOY_SSH_KEY</code>	Despliegue por SSH	<i>Settings</i> → <i>Secrets and variables</i> → <i>Actions</i>

Reglas seguidas:

- **Ningún secreto en el repositorio.** `.env` está en `.gitignore` ; solo se versiona `.env.example` .
- **Permisos mínimos.** El job `imagen` declara explícitamente `contents: read` y `packages: write` , en lugar del token con permisos amplios por defecto.
- Los secretos de despliegue se asocian al **environment**, no al repositorio, de modo que una rama cualquiera no puede leer las credenciales de producción.

6. Tiempos de ejecución aproximados

Etapa	Primera ejecución	Con caché
calidad	~50 s	~35 s
pruebas (por versión)	~70 s	~45 s
seguridad	~45 s	~30 s
imagen	~3 min	~40 s
smoke-test	~2 min	~1 min
Total (extremo a extremo)	~7 min	~3 min

El paralelismo de las tres primeras etapas ahorra alrededor de 2 minutos por ejecución.

7. Estrategia de ramas



Rama	Propósito	Protección recomendada
main	Código en producción	PR obligatorio, checks verdes, sin push directo
develop	Integración continua	Checks verdes
feature/*	Trabajo individual	—

Reglas a activar en *Settings* → *Branches* para **main**:

- Require a pull request before merging
- Require status checks to pass: **calidad**, **pruebas (20)**, **pruebas (22)**, **seguridad**
- Require branches to be up to date before merging

8. Cómo extender el pipeline

Objetivo	Cómo
Escaneo de vulnerabilidades de la imagen	Añadir <code>aquasecurity/trivy-action</code> tras el build
Pruebas end-to-end de navegador	Job con Playwright que ataque el contenedor de <code>smoke-test</code>
Análisis de calidad con SonarQube	Job que consuma <code>coverage/lcov.info</code> (ya se genera)
Despliegue canario	Rolling update en Kubernetes o dos servicios en Compose con pesos en el proxy
Notificaciones a Slack	Paso final con <code>slackapi/slack-github-action</code> y <code>if: always()</code>
Publicar releases	Job en tags <code>v*</code> con <code>softprops/action-gh-release</code>

9. Depuración de fallos del pipeline

Síntoma	Diagnóstico
<code>calidad</code> falla	Reproduzca en local con <code>npm run lint</code> ; <code>npm run lint:fix</code> corrige lo automatizable
<code>pruebas</code> falla solo en Node 22	Incompatibilidad de una dependencia con la versión nueva; revise el changelog
Cobertura por debajo del umbral	<code>npm run test:coverage</code> y abra <code>coverage/lcov-report/index.html</code>
<code>imagen</code> falla en <code>npm ci</code>	Lockfile desincronizado: ejecute <code>npm install</code> y haga commit de <code>package-lock.json</code>
<code>smoke-test</code> falla	El paso <code>docker logs</code> del job muestra el error real de la aplicación
El despliegue no se dispara	Verifique la condición <code>if: github.ref == 'refs/heads/main'</code> y desde qué rama vino el push

Manual de Operaciones — TaskFlow

Documento de referencia para quien opera el servicio en ejecución. Está escrito para consultarse **durante** un incidente: cada sección va del síntoma a la acción.

1. Fichas del servicio

Dato	Valor
Servicio	taskflow-api
Runtime	Node.js 20 sobre Debian slim
Puerto	3000
Base de datos	SQLite en /app/data/taskflow.db (volumen taskflow-data)
Logs	/app/logs/{app,error}.log (volumen taskflow-logs)
Liveness	GET /health
Readiness	GET /health/ready
Métricas	GET /metrics
Usuario del contenedor	node (sin privilegios)
Init	tini como PID 1

Consolas

Herramienta	URL
Aplicación	http://localhost:3000
Prometheus	http://localhost:9090
Alertas activas	http://localhost:9090/alerts
Alertmanager	http://localhost:9093
Grafana	http://localhost:3001

2. Operaciones cotidianas

Arrancar y detener

```
docker compose up -d           # arrancar todo
docker compose up -d app       # arrancar solo la aplicación
docker compose stop app        # detener sin borrar
docker compose restart app     # reiniciar
docker compose down            # bajar todo (conserva volúmenes)
docker compose down -v        # bajar y BORRAR los datos
```

Estado y logs

```
docker compose ps              # estado y salud
docker compose logs -f --tail=100 app  # logs en vivo
docker stats --no-stream        # CPU y memoria
docker inspect --format='{{.State.Health.Status}}' taskflow-app
```

Consultas rápidas de salud

```
curl -s localhost:3000/health | jq
curl -s localhost:3000/health/ready | jq
curl -s localhost:3000/api/tasks/stats | jq
curl -s localhost:3000/metrics | grep -E '^http_requests_total'
```

Actualizar a una versión nueva

```
# En un servidor, con la imagen publicada por el pipeline
export TASKFLOW_IMAGE=ghcr.io/edwindelossantos08/devops-final:latest
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d

# En desarrollo, reconstruyendo desde el código local
docker compose up -d --build

./scripts/smoke-test.sh http://localhost:3000 # verificación posterior
```

3. Runbooks de incidentes

Cada alerta de Prometheus apunta a una de estas secciones.

3.1 Aplicación caída

Alerta: `AplicacionCaída` · severidad **critical** · `up{job="taskflow-app"} == 0` durante 1 min

Impacto: el servicio no atiende ninguna petición.

Diagnóstico:

```
docker compose ps app          # ¿está corriendo o en restart loop?
docker compose logs --tail=200 app  # ¿qué dijo antes de morir?
docker inspect taskflow-app --format='{{.State.ExitCode}} {{.State.OOMKilled}}'
```

Código de salida	Significado	Acción
0	Apagado ordenado	Alguien lo detuvo: <code>docker compose up -d app</code>
1	Excepción no capturada	Busque el stack en los logs; véase §3.5
137	SIGKILL / OOM	Falta memoria: suba el límite o investigue la fuga (§3.4)
143	SIGTERM	Reinicio provocado por un despliegue: normal

Resolución:

```
docker compose up -d app
./scripts/wait-for-health.sh http://localhost:3000/health 60
./scripts/smoke-test.sh http://localhost:3000
```

Si vuelve a caer inmediatamente, haga **rollback** (§5) antes de seguir investigando: primero se restablece el servicio, después se busca la causa.

3.2 Tasa de errores 5xx alta

Alerta: `TasaDeErroresAlta` · severidad **critical** · `> 5 %` de respuestas 5xx durante 2 min

Diagnóstico — qué endpoint falla:

En Prometheus (<http://localhost:9090>):

```
sum by (route, status_code) (rate(http_requests_total{status_code=~"5.."}[5m]))
```

Ver los errores concretos en Grafana → Explore → Loki:

```
{job="taskflow"} | json | level="error"
```

Cada línea trae `requestId`, `path`, `message` y `stack`. Con el `requestId` se recupera la traza completa de esa petición:

```
{job="taskflow"} | json | requestId="<el-id>"
```

Causas frecuentes:

Causa	Señal	Acción
Base de datos inaccesible	<code>/health/ready</code> devuelve 503	Véase §3.3
Despliegue defectuoso	Los errores empiezan justo tras un deploy	Rollback (§5)
Disco lleno	<code>SQLITE_FULL</code> en los logs	<code>df -h</code> , libere espacio
Fuga de memoria	Sube junto con <code>MemoriaElevada</code>	Reinicio + §3.4

3.3 La base de datos no responde

Señal: `GET /health/ready` devuelve 503 con `{"checks":{"database":"down"}}`

Nótese que `/health` **sigue devolviendo 200**: es deliberado. Un fallo de base de datos saca la instancia de rotación (readiness) pero no reinicia el contenedor (liveness), porque reiniciar no arregla un volumen corrupto y sí pierde el diagnóstico.

Diagnóstico:

```
docker exec taskflow-app ls -la /app/data          # ¿existe el archivo?
docker exec taskflow-app sh -c 'df -h /app/data'   # ¿hay espacio?
docker volume inspect taskflow_taskflow-data
```

Verificar integridad:

```
docker exec taskflow-app node -e "
  const db = require('better-sqlite3')('/app/data/taskflow.db');
  console.log(db.pragma('integrity_check'));
"
```

Si el resultado no es `ok`, restaure desde copia de seguridad (§4.2).

3.4 Memoria elevada

Alerta: `MemoriaElevada` · severidad **warning** · RSS > 400 MB durante 5 min

Diagnóstico:

```
taskflow_process_resident_memory_bytes
taskflow_nodejs_heap_size_used_bytes
taskflow_nodejs_external_memory_bytes
```

Un heap que crece de forma **monótona** entre reinicios indica fuga; si sube y baja con el tráfico, es carga normal y lo que procede es subir el límite.

Mitigación inmediata:

```
docker compose restart app
```

Mitigación permanente — fijar el límite de heap y el del contenedor en `docker-compose.yml` :

```
services:
  app:
    environment:
      NODE_OPTIONS: '--max-old-space-size=384'
    deploy:
      resources:
        limits:
          memory: 512M
```

3.5 Latencia alta

Alerta: `LatenciaAlta` · severidad **warning** · p95 > 1 s durante 3 min

Localizar la ruta lenta:

```
histogram_quantile(0.95,
  sum by (le, route) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Ver las peticiones lentas concretas en Loki:


```
{job="taskflow"} | json | durationMs > 500
```

Causa	Verificación	Acción
CPU saturada	<code>rate(taskflow_process_cpu_seconds_total[5m])</code> cerca de 1	Escalar réplicas o subir CPU
Event loop bloqueado	<code>taskflow_nodejs_eventloop_lag_seconds</code> alto	Revisar operaciones síncronas pesadas
Tabla sin índice	Latencia proporcional al número de filas	Añadir índice en <code>src/db/database.js</code>
Contención de escritura en SQLite	Picos coincidentes con altas de tareas	WAL ya activo; considerar migrar a PostgreSQL

3.6 Exceso de errores 4xx

Alerta: `ExcesoDeErroresDeCliente` · severidad **warning** · > 5 req/s con 4xx durante 5 min

Normalmente **no** es un fallo del servidor, pero sí una señal:

```
sum by (route, status_code) (rate(http_requests_total{status_code=~"4.."}[5m]))
```

Código dominante	Interpretación
<code>400</code>	El frontend envía datos que la API rechaza: contrato desalineado tras un deploy
<code>404</code>	Enlaces rotos, o un escáner automatizado sondeando rutas
<code>429</code>	Rate limit activándose: tráfico legítimo en aumento o abuso

Para `429`, decida entre subir `RATE_LIMIT_MAX` (si el tráfico es legítimo) o mantener el límite y bloquear el origen en el proxy.

3.7 Reinicio del servicio

Alerta: `ReinicioDelServicio` · severidad **info**

Esperada tras un despliegue. Si **no** hubo despliegue, es un crash: revise el código de salida (§3.1) y busque `uncaughtException` en los logs.

4. Copias de seguridad

4.1 Realizar una copia

```
# Copia consistente del volumen completo
docker run --rm \
  -v taskflow_taskflow-data:/data:ro \
  -v "$(pwd)/backups:/backup" \
  alpine tar czf "/backup/taskflow-$(date +%F-%H%M).tar.gz" -C /data .
```

Para una copia en caliente sin detener el servicio, SQLite ofrece `VACUUM INTO` :

```
docker exec taskflow-app node -e "
  const db = require('better-sqlite3')('/app/data/taskflow.db');
  db.exec(`VACUUM INTO '/app/data/backup-${$(date +%F).db}`");
"
```

4.2 Restaurar

```
docker compose stop app

docker run --rm \
  -v taskflow_taskflow-data:/data \
  -v "$(pwd)/backups:/backup" \
  alpine sh -c 'rm -rf /data/* && tar xzf /backup/taskflow-FECHA.tar.gz -C /data'

docker compose start app
./scripts/smoke-test.sh http://localhost:3000
```

4.3 Política recomendada

Aspecto	Valor
Frecuencia	Diaria (cron a las 03:00)
Retención	7 diarias + 4 semanales
Ubicación	Fuera del host de la aplicación
Verificación	Restauración de prueba mensual

Una copia de seguridad **nunca probada** no es una copia de seguridad.

Cron sugerido:

```
0 3 * * * cd /opt/taskflow && ./scripts/backup.sh >> /var/log/taskflow-backup.log 2>&1
```

5. Rollback

El pipeline etiqueta cada imagen con el SHA corto del commit, lo que permite volver a una versión exacta.

El despliegue en servidor usa el override `docker-compose.prod.yml`, que sustituye la construcción local por la imagen publicada. Volver atrás es cambiar la etiqueta:

```
# 1. Identificar la versión estable anterior (por SHA del commit)
git log --oneline -10

# 2. Redesplegar esa imagen exacta
export TASKFLOW_IMAGE=ghcr.io/edwindelossantos08/devops-final:a1b2c3d
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d app

# 3. Verificar
./scripts/wait-for-health.sh http://localhost:3000/health 60
./scripts/smoke-test.sh http://localhost:3000
```

En un entorno de desarrollo donde el compose base **construye** la imagen en lugar de descargarla, el rollback equivalente es volver al commit y reconstruir:

```
git checkout a1b2c3d
docker compose up -d --build app
```

Criterio de decisión: si el servicio está degradado y la causa no se identifica en **10 minutos**, haga rollback primero e investigue después con el servicio ya restablecido.

Atención con los datos: el rollback revierte el código, no el esquema. Si la versión defectuosa aplicó una migración destructiva, hay que restaurar también desde copia (§4.2).

6. Escalado

SQLite escribe sobre un único archivo, por lo que **varias réplicas no pueden compartir el mismo volumen de escritura**. Camino de crecimiento:

Etapa	Situación	Acción
1	Una instancia saturada de CPU	Escalado vertical: más CPU al contenedor
2	Se necesita alta disponibilidad	Migrar a PostgreSQL y luego replicar la app
3	Tráfico alto sostenido	Balanceador + N réplicas sin estado + caché

La aplicación ya está preparada para el paso 2: todo el SQL está aislado en `src/services/tasks.service.js` y `src/db/database.js`.

7. Gestión de logs

Consultas frecuentes en Loki

```
{job="taskflow"} | json | level="error"           # solo errores
{job="taskflow"} | json | statusCode=~"5.."        # fallos de servidor
{job="taskflow"} | json | durationMs > 1000        # peticiones lentas
{job="taskflow"} | json | route="/api/tasks" | line_format "{{.method}} {{.statusCode}}
{{.durationMs}}ms"
sum(rate({job="taskflow"} | json | level="error" [5m])) # tasa de errores
```

Rotación

Winston rota a los 5 MB y conserva 5 archivos por tipo (25 MB máximo por archivo de log). Loki retiene 7 días (`retention_period: 168h`). Para ampliarlo, edite `monitoring/loki/loki-config.yml` y reinicie el contenedor.

Cambiar el nivel de log en caliente

```
docker compose up -d -e LOG_LEVEL=debug app
# recuerde volver a 'info' al terminar: 'debug' genera mucho volumen
```

8. Seguridad operativa

Medidas ya implementadas:

- Contenedor ejecutándose como usuario **no root** (`node`).
- **Helmet** con CSP restrictiva; `X-Powered-By` desactivado.
- **Rate limiting** en `/api` (300 req/min por IP por defecto).
- Límite de **100 kB** en el cuerpo de las peticiones.
- Consultas **parametrizadas** en todo el acceso a datos — sin concatenación de SQL.

- El frontend **escapa** todo texto antes de insertarlo en el DOM.
- Los mensajes de error 5xx se **enmascaran**: no se filtran detalles internos al cliente.
- `npm audit --audit-level=high` bloqueante en el pipeline.

Pendientes recomendados antes de una producción real:

- ☐ Terminar TLS en un proxy inverso (§6.4 de [INSTALACION.md](#))
- ☐ Restringir `/metrics` a la red interna
- ☐ Cambiar las credenciales por defecto de Grafana
- ☐ Autenticación y autorización de usuarios en la API
- ☐ Escaneo de la imagen con Trivy en el pipeline

9. Checklists

Antes de desplegar

- ☐ El pipeline está verde en la rama de origen
- ☐ Existe una copia de seguridad reciente
- ☐ Se conoce el SHA de la versión actual (para rollback)
- ☐ Se avisó al equipo si la ventana no es de bajo tráfico

Después de desplegar

- ☐ `/health` y `/health/ready` responden 200
- ☐ `./scripts/smoke-test.sh` pasa completo
- ☐ El target de Prometheus está **UP**
- ☐ No hay alertas nuevas en <http://localhost:9090/alerts>
- ☐ La tasa de error y la latencia p95 se mantienen en los valores previos (10 min de observación)

Revisión semanal

- ☐ Revisar alertas disparadas durante la semana y ajustar umbrales ruidosos
 - ☐ Comprobar que las copias de seguridad se están generando
 - ☐ `npm audit` sin vulnerabilidades altas nuevas
 - ☐ Uso de disco de los volúmenes de Prometheus y Loki
 - ☐ Tendencia de latencia y memoria a 7 días
-

10. Escalado de incidentes

Severidad	Ejemplo	Respuesta	Notificación
Crítica	Servicio caído, > 5 % de 5xx	Inmediata	Guardia (repetición cada 30 min)
Warning	Latencia p95 > 1 s, memoria alta	En horario laboral	Canal del equipo (cada 4 h)
Info	Reinicio del servicio	Solo registro	Sin notificación activa

Alertmanager aplica una regla de **inhibición**: cuando `AplicacionCaida` está activa, se suprimen las alertas `warning` e `info` del mismo servicio. Un servicio caído genera una sola página, no seis.

Presentación Final — TaskFlow

Guion de la presentación, demo en vivo y análisis del proyecto. Autor: **Edwin De Los Santos · 2025-1032** Duración objetivo: **15 minutos de exposición + 5 de preguntas.**

Estructura de la presentación

#	Sección	Tiempo
1	Introducción y objetivos	1 min
2	Arquitectura de la aplicación	2 min
3	Pipeline CI/CD	3 min
4	Demo en vivo	5 min
5	Monitoreo y alertas	2 min
6	Desafíos y lecciones aprendidas	2 min
7	Preguntas	5 min

Diapositiva 1 — Portada

TaskFlow — Pipeline DevOps de extremo a extremo Práctica Final · Edwin De Los Santos · 2025-1032

Frase de apertura: *"La aplicación es sencilla a propósito. Lo que construimos no es un gestor de tareas: es todo el camino automatizado que lleva un commit a producción con confianza."*

Diapositiva 2 — El problema

Sin pipeline:

- El despliegue es manual, distinto cada vez y depende de quién lo haga.
- Los errores se descubren **en producción**, no antes.
- No hay forma de saber si el servicio está sano hasta que un usuario reclama.
- Volver atrás es improvisar.

Con pipeline: **cada commit atraviesa las mismas seis puertas de calidad, siempre.**

Diapositiva 3 — Arquitectura de la aplicación

```
Navegador (HTML/CSS/JS)
  | HTTP/JSON
  ▼
Express — rutas → controladores → servicio → SQLite
  |
  ├── métricas Prometheus (/metrics)
  └── logs JSON (Winston)
```

Puntos a destacar:

- Arquitectura **en capas**: los controladores no ejecutan SQL, el servicio no conoce Express.
- Esa separación es lo que permite tener **31 pruebas unitarias sin levantar un servidor**.
- `app.js` construye la aplicación; `server.js` la arranca. Supertest monta `app` directamente.

Diapositiva 4 — Stack tecnológico

Capa	Herramienta
Aplicación	Node.js 20 + Express + SQLite
Pruebas	Jest + Supertest (54 pruebas, 92 % cobertura)
Calidad	ESLint 9 + Prettier + npm audit
Contenedores	Docker multi-etapa + Compose (6 servicios)
CI/CD	GitHub Actions (8 jobs)
Métricas	prom-client + Prometheus
Logs	Winston + Promtail + Loki
Visualización	Grafana
Alertas	Alertmanager (6 reglas)

Diapositiva 5 — El pipeline

```
commit → [calidad || pruebas || seguridad] → imagen → smoke-test → despliegue
```

Tres decisiones que vale la pena defender:

1. **Las tres validaciones corren en paralelo**. Feedback de fallo en ~1 min en lugar de ~3. En un pipeline se optimiza el tiempo hasta el *fallo*, no hasta el éxito.

2. **Los pull requests validan pero no publican.** Código propuesto no es código aceptado; no debe poder inyectar artefactos en el registro antes de la revisión.
3. **Los smoke tests corren contra el contenedor real.** Jest valida el código; no detecta que el `Dockerfile` olvidó copiar `public/`. Son verificaciones de naturaleza distinta.

Diapositiva 6 — Guion de la demo en vivo

Preparación previa (antes de la presentación, no en vivo):

```
docker compose up -d --build      # que el stack ya esté caliente
node scripts/seed.js              # datos de ejemplo
```

Tenga abiertas en pestañas: la app, GitHub Actions, Prometheus/alerts, Grafana.

Paso 1 — La aplicación funciona (45 s)

<http://localhost:3000> — crear una tarea, marcarla completada, ver el contador actualizarse.

"Frontend, API y base de datos. Es la parte simple. Ahora veamos lo que hay detrás."

Paso 2 — Las pruebas (45 s)

```
npm test
```

"54 pruebas en dos segundos y medio. Unitarias contra la lógica de negocio, de integración contra la API completa vía HTTP."

Paso 3 — La puerta de calidad se cierra (90 s) — el momento clave

Rompa algo a propósito, en vivo:

```
# Introducir un fallo real en la validación
# (en macOS: sed -i '' ...   en Linux: sed -i ... )
sed -i '' "s/if (!titulo) {/if (false) {/" src/services/tasks.service.js
npm test
```

Alternativa sin `sed`, más segura en vivo: abrir `src/services/tasks.service.js`, comentar la línea `throw new AppError('El título es obligatorio.', 400);` y guardar.

"Cinco pruebas fallan de inmediato. Este commit no llega a construirse ni a desplegarse: el job `imagen` depende de que `pruebas` pase."

Revertir:

```
git checkout src/services/tasks.service.js
npm test    # verde otra vez
```

Paso 4 — El pipeline en GitHub (60 s)

Muestre una ejecución real: los tres jobs en paralelo, el fan-in hacia `imagen`, el `smoke-test` y la tabla del resumen.

Paso 5 — Métricas en vivo (60 s)

Genere tráfico y muestre Grafana reaccionando:

```
for i in $(seq 1 40); do
  curl -s -X POST localhost:3000/api/tasks -H 'Content-Type: application/json' \
    -d '{"title":"'Demo $i'"}' > /dev/null
done
```

"Petición por segundo, latencia p95, tareas creadas por minuto. Métricas técnicas y de negocio en el mismo panel."

Paso 6 — Una alerta real (60 s) — el cierre fuerte

```
docker compose stop app
```

- <http://localhost:9090/alerts> → `AplicacionCaida` pasa a **PENDING** y, tras 1 minuto, a **FIRING**.
- <http://localhost:9093> → la alerta llega agrupada a Alertmanager.

"Un minuto de espera es deliberado: evita alertar por un reinicio de dos segundos."

```
docker compose start app    # la alerta se resuelve sola
```

Paso 7 — Los logs (30 s)

Grafana → Explore → Loki:

```
{job="taskflow"} | json | level="error"
```

"Cada línea trae un requestId que también viaja en la cabecera de la respuesta: un usuario reporta un error, pegamos su ID y tenemos la traza exacta."

Diapositiva 7 — Monitoreo: los tres pilares

Pilar	Herramienta	Pregunta que responde
Métricas	Prometheus + Grafana	¿Cómo se comporta el sistema?
Logs	Winston + Loki	¿Qué pasó exactamente en esta petición?
Alertas	Alertmanager	¿Cuándo debo intervenir?

Detalle técnico que demuestra criterio — cardinalidad:

Las rutas se normalizan a `/api/tasks/:id` en lugar de `/api/tasks/42`. Sin esa normalización, mil tareas generarían mil series temporales distintas y Prometheus se degradaría. La misma lógica se aplicó en Promtail: solo `level`, `route` y `statusCode` se promueven a etiquetas de Loki; `requestId` se queda como campo.

Diapositiva 8 — Desafíos y soluciones

#	Desafío	Solución adoptada
1	Las pruebas se contaminaban entre sí: una suite dejaba datos que rompían a la siguiente	<code>NODE_ENV=test</code> fuerza SQLite en <code>:memory:</code> y <code>resetDb()</code> corre en cada <code>beforeEach</code>
2	GET /api/tasks/stats devolvía 400: Express la capturaba como <code>/:id</code> con id <code>"stats"</code>	Declarar la ruta literal antes que la paramétrica en el router
3	Cardinalidad explosiva en Prometheus: cada id generaba una serie nueva	Normalizar la etiqueta <code>route</code> usando <code>req.route.path</code>
4	La imagen Docker pesaba más de 1 GB: incluía compiladores y devDependencies	Build multi-etapa: la etapa <code>runtime</code> solo recibe <code>node_modules</code> de producción y el código
5	docker stop cortaba peticiones en vuelo: Node no recibía <code>SIGTERM</code> como PID 1	<code>tini</code> como init + <code>server.close()</code> con temporizador de 10 s
6	Alerta de caída con seis avisos simultáneos: latencia, memoria y errores se disparaban a la vez	Regla de inhibición en Alertmanager sobre <code>AplicacionCaída</code>
7	La readiness reiniciaba el contenedor ante un fallo de BD	Separar liveness (<code>/health</code> , no toca la BD) de readiness (<code>/health/ready</code> , sí)
8	La cobertura bajaba sin que nadie lo notara	<code>coverageThreshold</code> en <code>jest.config.js</code> hace fallar la build

Diapositiva 9 — Lecciones aprendidas

1. **Automatizar temprano cuesta menos que automatizar tarde.** Montamos el pipeline con la aplicación a medias. Cada funcionalidad posterior nació ya validada; no hubo una fase de "ahora hay que hacer que todo esto pase las pruebas".
2. **La observabilidad se diseña, no se añade.** El `requestId` que correlaciona logs, la normalización de rutas y la separación liveness/readiness son decisiones de diseño de la aplicación, no configuración del stack de monitoreo. Añadirlas después habría implicado tocar todos los middlewares.
3. **Las pruebas de humo cubren un hueco que las unitarias no ven.** El primer contenedor que construimos arrancaba y pasaba `npm test` , pero devolvía 404 en `/` porque el `.dockerignore` excluía `public/` . Solo un smoke test contra el contenedor real detecta esa clase de fallo.
4. **Una alerta ruidosa es peor que ninguna alerta.** Nuestra primera versión disparaba `LatenciaAlta` en cada despliegue. Los `for` de 1 a 5 minutos y las reglas de inhibición son la diferencia entre un sistema que se consulta y uno que se silencia.
5. **La documentación operativa se escribe pensando en el incidente.** `OPERACIONES.md` va del síntoma a la acción porque a las 3 de la madrugada nadie lee una explicación de arquitectura: busca el comando.

Diapositiva 10 — Cumplimiento del enunciado

Requisito	Evidencia
Repositorio Git	Ramas <code>main / develop</code> , <code>.gitignore</code> , historial limpio
Pipeline CI/CD	<code>.github/workflows/ci-cd.yml</code> — 8 jobs
Contenedores Docker	<code>Dockerfile</code> multi-etapa + <code>docker-compose.yml</code> con 6 servicios
Pruebas unitarias	31 pruebas en <code>tests/unit/</code>
Pruebas de integración	23 pruebas en <code>tests/integration/</code>
Análisis estático	ESLint 9 + Prettier + <code>npm audit</code> , bloqueantes en CI
Logs centralizados	Winston JSON → Promtail → Loki → Grafana
Métricas	7 familias de métricas → Prometheus → dashboard de 11 paneles
Alertas	6 reglas con severidad, agrupación e inhibición
Documentación	README + Instalación + Pipeline + Operaciones + Presentación

Diapositiva 11 — Trabajo futuro

- Escaneo de la imagen con **Trivy** en el pipeline.
- Pruebas **end-to-end** de navegador con Playwright.
- Migración a **PostgreSQL** para permitir múltiples réplicas.
- **Despliegue canario** con incremento progresivo de tráfico.
- **Trazas distribuidas** con OpenTelemetry, completando el tercer pilar de la observabilidad.
- **Autenticación** de usuarios y autorización por rol.

Preguntas frecuentes previstas

¿Por qué SQLite y no MongoDB o PostgreSQL? El enunciado admite SQLite y aporta una ventaja concreta para este proyecto: cero servicios externos, lo que mantiene el foco en el pipeline. Además, todo el SQL está aislado en dos archivos, por lo que migrar a PostgreSQL cuando haga falta escalar horizontalmente es un cambio acotado.

¿Qué pasa si falla el despliegue en producción? Rollback por SHA: el pipeline etiqueta cada imagen con el hash del commit, así que se vuelve a una versión exacta con `APP_VERSION=<sha> docker compose up -d` . El criterio operativo es: si no se identifica la causa en 10 minutos, primero se revierte.

¿Por qué liveness y readiness separados? Liveness responde "¿el proceso está vivo?" y readiness "¿puede atender tráfico?". Si la base de datos falla, queremos sacar la instancia de rotación (readiness en 503), no reiniciar el contenedor: reiniciar no arregla un volumen corrupto y destruye el estado que

permitiría diagnosticarlo.

¿Cómo evitan las alertas falsas? Con tres mecanismos: la cláusula `for` (la condición debe sostenerse entre 1 y 5 minutos), las reglas de inhibición (una caída suprime las alertas derivadas) y la agrupación por `alertname` + `service` en Alertmanager.

¿La cobertura del 92 % garantiza que no hay errores? No. La cobertura mide qué líneas se ejecutan, no si las aserciones son correctas. Por eso el pipeline combina cobertura con pruebas de integración sobre HTTP real y smoke tests contra el contenedor: son tres formas distintas de equivocarse.

¿Cuánto tarda el pipeline completo? Unos 7 minutos la primera vez y cerca de 3 con la caché de npm y de Docker buildx caliente. El feedback de un fallo de lint o de pruebas llega en menos de 1 minuto.